

SQLite Commands for Online Voting System Database

Introduction

The **Online Voting System** utilizes **SQLite**, a lightweight, serverless database, to manage data for users, candidates, and votes in the `election.db` database. The schema comprises three tables: `users`, `candidates`, and `votes`, which support core functionalities such as user registration, candidate management, vote tracking, and result visualization. This document provides a comprehensive set of simple SQLite commands for performing essential database operations, including table creation, data insertion, querying, updating, deletion, and maintenance. Each command is accompanied by its purpose, syntax, example, and technical insights, tailored to the project's schema. The commands are designed for developers, database administrators, and system architects, offering a practical guide to manage the database effectively.

1. Database Setup and Table Creation

Overview

These commands initialize the `election.db` database and create the `users`, `candidates`, and `votes` tables, establishing the structure for storing application data. They align with the `init_db` function in the project's Python code.

Commands

- **Create the `users` Table:**

```
CREATE TABLE IF NOT EXISTS users (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    username TEXT UNIQUE NOT NULL,  
    password TEXT NOT NULL,  
    full_name TEXT,  
    gender TEXT,  
    has_voted BOOLEAN DEFAULT FALSE,
```

```
face_encoding_path TEXT
);
```

- **Purpose:** Defines a table to store user details, including a unique ID, username, hashed password, full name, gender, voting status, and path to facial recognition encoding files.
- **Details:**
 - `PRIMARY KEY AUTOINCREMENT` : Generates unique, sequential IDs.
 - `UNIQUE NOT NULL` on `username` : Ensures no duplicate usernames and requires a value.
 - `DEFAULT FALSE` on `has_voted` : Initializes new users as non-voters.
 - `face_encoding_path` : Stores file paths (e.g., `face_encodings/john_doe.npy`) for biometric data.
- **Example Use:** Execute during database setup to enable user registration.
- **Technical Insight:** The `IF NOT EXISTS` clause ensures idempotency, preventing errors if the table already exists. `AUTOINCREMENT` adds a minor overhead but guarantees unique IDs even after deletions.

- **Create the `candidates` Table:**

```
CREATE TABLE IF NOT EXISTS candidates (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT UNIQUE NOT NULL,
    description TEXT,
    votes INTEGER DEFAULT 0
);
```

- **Purpose:** Creates a table for candidates, storing their ID, unique name, optional description, and vote count.
- **Details:**
 - `UNIQUE NOT NULL` on `name` : Prevents duplicate candidates.
 - `DEFAULT 0` on `votes` : Initializes vote counts to zero.
 - `description` : Optional field for candidate details, can be `NULL`.
- **Example Use:** Run to prepare the database for candidate data, used by admins via the admin panel.
- **Technical Insight:** The `INTEGER` type for `votes` supports up to $2^{63}-1$ votes, sufficient for most elections. A `CHECK(votes >= 0)` constraint could further ensure data integrity.

- **Create the `votes` Table:**

```
CREATE TABLE IF NOT EXISTS votes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    candidate_id INTEGER NOT NULL,
    timestamp TEXT NOT NULL,
    FOREIGN KEY(user_id) REFERENCES users(id),
    FOREIGN KEY(candidate_id) REFERENCES candidates(id)
);
```

- **Purpose:** Tracks votes by linking users to candidates with a timestamp.
- **Details:**
 - **FOREIGN KEY** constraints: Ensure `user_id` and `candidate_id` reference valid records in `users` and `candidates`.
 - **NOT NULL**: Mandates values for `user_id`, `candidate_id`, and `timestamp`.
 - **timestamp**: Stores ISO 8601 strings (e.g., `2025-04-21T11:45:00`).
- **Example Use:** Execute to enable vote recording, used in the `cast_vote` function.
- **Technical Insight:** SQLite requires `PRAGMA foreign_keys = ON;` to enforce foreign keys, which must be set per connection. Without this, invalid IDs could be inserted, breaking referential integrity.

Significance

- These commands establish the relational structure of the database, enabling secure storage and retrieval of election data.
 - They are critical for the application's initialization, ensuring tables are ready for use by the Python code.
-

2. Inserting Data

Overview

Insert commands add records to the tables, supporting user registration, candidate creation, and vote recording. They correspond to the project's `register_user`, `add_candidate`, and `cast_vote` functions.

Commands

- **Insert a New User:**

```
INSERT INTO users (username, password, full_name, gender, has_voted,
VALUES ('john_doe', 'a665a45920422f9d417e4867efdc4fb8a04a1f3fff1fa07e
```

- **Purpose:** Adds a user to the `users` table.
- **Details:**
 - `password` : A SHA-256 hash (e.g., for “password123”).
 - `has_voted` : Set to `FALSE` for new users.
 - `face_encoding_path` : References a `.npy` file storing the user’s facial encoding.
- **Example Use:** Executed when a user completes the registration form and face capture.
- **Technical Insight:** The Python code uses parameterized queries (`c.execute("INSERT ... VALUES (?, ?, ?, ?, ?, ?)", values)`) to prevent SQL injection. A `UNIQUE` violation on `username` raises `sqlite3.IntegrityError` , handled in the code.

- **Insert a New Candidate:**

```
INSERT INTO candidates (name, description, votes)
VALUES ('Jane Smith', 'Experienced leader with a vision for change',
```

- **Purpose:** Adds a candidate to the `candidates` table.
- **Details:**
 - `name` : Must be unique and non-null.
 - `description` : Optional, can be `NULL` (e.g., `INSERT INTO candidates (name, votes) VALUES ('John Lee', 0);`).
 - `votes` : Initialized to 0.
- **Example Use:** Run by admins via the **Admin** tab to add candidates.
- **Technical Insight:** Duplicate `name` values trigger `sqlite3.IntegrityError` , caught in the `add_candidate` function to display an error message.

- **Insert a Vote:**

```
INSERT INTO votes (user_id, candidate_id, timestamp)
VALUES (1, 1, '2025-04-21T11:45:00');
```

- **Purpose:** Records a vote linking a user to a candidate.
- **Details:**
 - `user_id` and `candidate_id` : Must reference existing IDs in `users` and `candidates` .
 - `timestamp` : ISO 8601 format, generated by `datetime.now().isoformat()` in Python.

- **Example Use:** Executed in `cast_vote` when a user submits their vote.
- **Technical Insight:** Foreign key constraints (if enabled) prevent invalid IDs. The Python code checks `has_voted` before insertion to enforce one vote per user.

Significance

- Insert commands populate the database, enabling the core functionalities of user registration, candidate management, and voting.
 - They ensure data consistency through constraints and error handling in the application.
-

3. Querying Data

Overview

Select commands retrieve data for displaying user profiles, candidate lists, voting results, and audit reports. They support the project's `get_user_profile`, `get_candidates`, and `get_results` functions.

Commands

- **Retrieve All Users:**

```
SELECT id, username, full_name, gender, has_voted
FROM users;
```

- **Purpose:** Fetches all user records for reporting or administration.
- **Details:**
 - Excludes `password` and `face_encoding_path` for security.
 - Returns columns relevant for user management.
- **Example Use:** Useful for admins to view registered users or generate statistics.
- **Technical Insight:** This query is lightweight but could be slow for millions of users. Adding `LIMIT 100` or pagination (`OFFSET 100`) would optimize large datasets.

- **Get a Specific User's Profile:**

```
SELECT username, full_name, gender, has_voted
FROM users
WHERE id = 1;
```

- **Purpose:** Retrieves details for a specific user by their ID.
- **Details:** Used in the `get_user_profile` function to populate the **Profile** tab.
- **Example Use:** Run when a user views or edits their profile.
- **Technical Insight:** The `id` column is indexed (as the primary key), making this query highly efficient.

- **List All Candidates:**

```
SELECT id, name, description, votes
FROM candidates;
```

- **Purpose:** Fetches all candidates for voting or result display.
- **Details:** Used in `get_candidates`, cached with `@st.cache_data` to reduce database queries.
- **Example Use:** Populates the **Vote** tab's candidate dropdown and **Results** tab's metrics.
- **Technical Insight:** Caching improves performance, but cache invalidation requires manual clearing (`st.cache_data.clear()`) after candidate updates.

- **Count Total Votes:**

```
SELECT SUM(votes)
FROM candidates;
```

- **Purpose:** Calculates the total votes cast across all candidates.
- **Details:** Used in `get_results` to compute vote percentages (`votes / total_votes * 100`).
- **Example Use:** Displays total votes in the **Results** tab.
- **Technical Insight:** `SUM` is efficient for small datasets but could be optimized with a materialized view for large-scale elections.

- **Get Voting History:**

```
SELECT u.username, c.name, v.timestamp
FROM votes v
JOIN users u ON v.user_id = u.id
JOIN candidates c ON v.candidate_id = c.id
ORDER BY v.timestamp DESC;
```

- **Purpose:** Retrieves a detailed history of votes, linking users to candidates.
- **Details:**
 - Uses `JOIN` to combine `users`, `candidates`, and `votes`.

- `ORDER BY timestamp DESC` sorts by most recent votes.
- **Example Use:** Useful for auditing or generating election reports.
- **Technical Insight:** Joins can be slow for large datasets. Creating indexes on `votes.user_id` and `votes.candidate_id` (`CREATE INDEX idx_votes_user_id ON votes(user_id);`) improves performance.

Significance

- Query commands enable dynamic data retrieval, supporting user interaction and transparency in the voting process.
 - They are optimized for the project's small-scale use case but can be enhanced for larger datasets with indexing.
-

4. Updating Data

Overview

Update commands modify records, such as marking users as voted, incrementing candidate votes, or updating profiles. They align with the `cast_vote` and `update_user_profile` functions.

Commands

- **Mark a User as Voted:**

```
UPDATE users
SET has_voted = TRUE
WHERE id = 1;
```

- **Purpose:** Updates a user's voting status after casting a vote.
- **Details:** Sets `has_voted` to `TRUE` for the specified `id`.
- **Example Use:** Executed in `cast_vote` to enforce one vote per user.
- **Technical Insight:** The `id` index ensures fast updates. The Python code checks `has_voted` before updating to avoid redundant operations.

- **Increment a Candidate's Vote Count:**

```
UPDATE candidates
SET votes = votes + 1
WHERE id = 1;
```

- **Purpose:** Increments the vote count for a candidate.
- **Details:** Uses `votes = votes + 1` for atomic increment.
- **Example Use:** Run in `cast_vote` when a user selects a candidate.
- **Technical Insight:** SQLite's atomic updates prevent race conditions in concurrent environments, but high-frequency updates may benefit from a transaction.

- **Update a User's Profile:**

```
UPDATE users
SET username = 'new_username', full_name = 'New Name', gender = 'Female'
WHERE id = 1;
```

- **Purpose:** Modifies a user's profile details.
- **Details:**
 - Validates `username` uniqueness to avoid `sqlite3.IntegrityError`.
 - Only specified fields are updated, preserving others.
- **Example Use:** Used in `update_user_profile` when a user edits their profile.
- **Technical Insight:** The Python code uses parameterized queries to safely update fields, preventing SQL injection.

Significance

- Update commands ensure the database reflects user actions, such as voting or profile changes.
 - They maintain data integrity through constraints and application-level validation.
-

5. Deleting Data

Overview

Delete commands remove records, primarily used for resetting votes during election resets. They align with the `reset_votes` function, though user deletion is not implemented in the provided code.

Commands

- **Delete All Votes:**

```
DELETE FROM votes;
```


- **Purpose:** Clears all vote records from the `votes` table.
- **Details:** Used in `reset_votes` to reset the election without affecting users or candidates.
- **Example Use:** Run by admins via the **Admin** tab's reset feature.
- **Technical Insight:** `DELETE` is efficient for small tables but may slow with millions of records. `TRUNCATE TABLE votes;` (not supported in SQLite) or partitioning could optimize large-scale resets.

- **Reset Candidate Votes:**

```
UPDATE candidates
SET votes = 0;
```

- **Purpose:** Resets vote counts for all candidates to zero.
- **Details:** Preserves candidate records, unlike deletion. Part of `reset_votes`.
- **Example Use:** Executed during a vote reset to restart the election.
- **Technical Insight:** An `UPDATE` is used instead of `DELETE` to maintain candidate metadata, reducing the need to reinsert candidates.

- **Delete a Specific User** (Proposed Extension):

```
DELETE FROM users
WHERE id = 1;
```

- **Purpose:** Removes a user by ID (not in the provided code but useful).
- **Details:**
 - Requires `ON DELETE CASCADE` on `votes` foreign keys to remove related votes.
 - Without cascading, manual deletion of `votes` records is needed first.
- **Example Use:** Could be implemented for account deletion functionality.
- **Technical Insight:** Cascading deletes simplify cleanup but require careful configuration to avoid unintended data loss.

Significance

- Delete commands support administrative tasks like election resets, ensuring the database can be reused for multiple voting cycles.
- The backup mechanism in `reset_votes` (using `shutil.copy`) ensures data recovery, enhancing reliability.

6. Database Maintenance

Overview

Maintenance commands optimize performance, ensure integrity, and manage the database's health, supporting long-term operation.

Commands

- **Enable Foreign Keys:**

```
PRAGMA foreign_keys = ON;
```

- **Purpose:** Activates foreign key constraints for referential integrity.
- **Details:** Must be executed at the start of each database connection, as SQLite disables foreign keys by default.
- **Example Use:** Ensures `votes` table references valid `user_id` and `candidate_id`.
- **Technical Insight:** The Python code should include this in `init_db` to enforce constraints consistently.

- **Vacuum the Database:**

```
VACUUM;
```

- **Purpose:** Reclaims unused space and defragments the database file.
- **Details:** Reduces file size after deletions (e.g., vote resets) and improves query performance.
- **Example Use:** Run periodically or after large deletions to optimize storage.
- **Technical Insight:** `VACUUM` can be slow for large databases (>1GB), so schedule it during low-traffic periods.

- **Check Database Integrity:**

```
PRAGMA integrity_check;
```

- **Purpose:** Verifies the database for corruption.
- **Details:** Returns `ok` if intact or error details if corrupted (e.g., due to disk failures).
- **Example Use:** Run before backups or after crashes to ensure data reliability.
- **Technical Insight:** Combine with `PRAGMA quick_check;` for faster but less thorough checks in production.

- **Create an Index for Performance:**

```
CREATE INDEX idx_votes_user_id ON votes (user_id);
```

- **Purpose:** Speeds up queries involving `user_id` in `votes` (e.g., voting history joins).
- **Details:**
 - Indexes improve read performance for `SELECT` and `JOIN` queries.
 - Slightly increases insert/update time and storage.
- **Example Use:** Add for large datasets to optimize queries like the voting history command.
- **Technical Insight:** Consider additional indexes (e.g., `idx_votes_candidate_id`) based on query patterns, but avoid over-indexing to minimize write overhead.

Significance

- Maintenance commands ensure the database remains efficient, reliable, and scalable.
 - They support the project's operational needs, particularly for frequent vote resets or growing user bases.
-

Conclusion

This document provides a concise yet comprehensive set of SQLite commands for managing the **Online Voting System's** `election.db` database. Covering table creation, data insertion, querying, updating, deletion, and maintenance, these commands align with the project's Python functions (`init_db`, `register_user`, `cast_vote`, `get_results`, etc.) and support its core functionalities. By leveraging SQLite's features like foreign keys, indexes, and parameterized queries, the commands ensure data integrity, security, and performance. For advanced use cases, consider extending the schema with audit logs, user deletion, or partitioning for large-scale elections. These commands empower developers and administrators to operate the database effectively, enabling a secure and efficient online voting experience.
